

LABORATOR 4

Comunicarea între procese: Semnale

1. Scopul lucrării

În această lucrare ne propunem să analizăm cum se poate face uz de facilitățile UNIX referitoare la *semnale* pentru a realiza comunicarea între procese.

2. Considerații teoretice

Semnalele sunt o modalitate de exprimare a evenimentelor care apar asincron în sistem. Un proces oarecare poate atât să genereze, cât și să primească semnale. În cazul în care un proces primește un semnal, el poate alege să reacționeze la semnalul respectiv într-unul din următoarele trei moduri:

- Să le capteze, executând o acțiune oarecare, prin intermediul unei **funcții de tratare a semnalului (signal handler)**
- Să le ignore
- Să execute acțiunea implicită la primirea unui semnal, care poate fi, după caz, terminarea procesului sau ignorarea semnalului respectiv.

Semnalele pot fi de mai multe tipuri, care corespund în general unor acțiuni specifice. Fiecare semnal are asociat un număr, iar acestor numere le corespund unele constante simbolice definite în bibliotecile sistemului de operare. Standardul POSIX.1 definește câteva semnale care trebuie să existe în orice sistem UNIX. Aceste semnale sunt cuprinse în următoarea listă:

SIGHUP	1	<u>Hangup</u> - terminalul folosit de proces a fost închis
SIGINT	2	<u>Interrupt</u> - semnifică în mod uzual întreruperea procesului prin Ctrl-C
SIGQUIT	3	<u>Quit</u> - are semnificația unei cereri de ieșire din program a utilizatorului
SIGILL	4	<u>Illegal Instruction</u> - se generează atunci când procesul a executat o instrucțiune al cărei <i>opcode</i> nu are corespondent în setul de instrucțiuni sau pentru care nu există privilegii suficiente
SIGABRT	6	<u>Abort</u> - semnal de terminare anormală a procesului generat de abort(3)
SIGFPE	8	<u>Floating Point Exception</u> - semnal generat de o eroare în virgula mobilă (de exemplu, în cazul unei împărțiri cu zero)
SIGKILL	9	<u>Kill</u> - semnal care are ca efect distrugerea imediată a procesului
SIGUSR1, 10,	<u>User Defined 1, 2</u>	- semnale a căror generare și tratare sunt

SIGUSR2	12	lasate in seama utilizatorului, folosite de obicei pentru generarea de diverse actiuni in logica unui proces
SIGSEGV	11	<u>Segmentation Fault</u> - se genereaza atunci cand un proces a facut referinta la o adresa de memorie invalida
SIGPIPE	13	<u>Broken Pipe</u> - se genereaza atunci cand s-a incercat scrierea intr-un <i>pipe</i> care are descriptorul de la capatul de citire inchis
SIGALRM	14	<u>Timer Alarm</u> - semnal generat in urma expirarii timer-ului pozitionat de alarm(2)
SIGTERM	15	<u>Terminate</u> - Semnifica o cerere de terminare normala a procesului
SIGCONT	18	<u>Continue</u> - Are ca rezultat continuarea unui proces oprit prin SIGSTOP
SIGSTOP	19	<u>Stop</u> - Are ca rezultat suspendarea executiei procesului pana cand aceasta va fi reluata prin primirea unui semnal SIGCONT

Diferitele variante de UNIX existente implementeaza aceste semnale, adaugand si altele noi. Pentru a obtine o lista cu toate semnalele existente in Linux, de exemplu, consultati pagina de manual **signal(7)**.

2. Tratarea semnalelor

2.1 Apelul sistem signal()

Modalitatea ANSI prin care se poate realiza captarea semnalelor in scopul tratarii lor este data de apelul sistem **signal()**. Acest apel sistem are urmatoarea forma:

```
#include <signal.h>
```

```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signal, sighandler_t handler);
```

Executia acestui apel sistem va face ca, pentru semnalul cu numarul signal specificat ca si parametru, sa se instaleze o noua rutina de tratare specificata prin parametrul handler. Acest parametru trebuie sa aiba tipul *sighandler_t*, care inseamna de fapt ca trebuie sa fie adresa unei functii care nu returneaza nimic si are un singur parametru formal de tip intreg. Acest parametru va primi ca valoare, in momentul executiei rutinei de tratare a semnalului, numarul semnalului care se trateaza. Acest lucru este util, de exemplu, atunci cand se doreste sa se trateze mai multe tipuri de semnale folosind aceeasi rutina.

Exista doua valori speciale pentru parametrul handler, si anume: **SIG_IGN**, care instruieste procesul sa ignore semnalul specificat, si **SIG_DFL**, care reseteaza tratarea semnalului la comportamentul implicit.

Apelul sistem **signal()** returneaza rutina anterioara de tratare a semnalului, sau **SIG_ERR** in caz de eroare. Acest lucru se poate intampla, de exemplu, cand semnalul cu numarul specificat nu exista, sau cand rutina de tratare specificata nu are tipul corespunzator. Tot **SIG_ERR** se returneaza si atunci cand signal este **SIGKILL** sau **SIGSTOP**, care sunt semnale ce nu pot fi nici captate, nici ignorate.

Desi marele avantaj al apelului sistem **signal()** asupra altor metode de captare a semnalelor este simplitatea, folosirea lui are si cateva dezavantaje. Unul dintre acestea este portabilitatea redusa, care apare din cauza ca diversele implementari ale acestui apel sistem in diverse sisteme UNIX

se comporta in mod diferit in cazul in care se capteaza semnalul. De exemplu, implementarea originala UNIX (folosita si de sistemele UNIX System V si Linux libc4 si libc5) este aceea ca, dupa o executie a rutinei de tratare, aceasta este resetata la valoarea **SIG_DFL**, ceea ce inseamna ca rutina trebuie sa apeleze **signal()** inainte de terminarea ei pentru a se "reinstala" in vederea tratarii urmatoarei aparitii a semnalului. Pe de alta parte, sistemele din familia 4.3BSD (si Linux glibc2) nu reseteaza rutina de tratare, dar blocheaza aparitiile semnalului respectiv in timpul executiei rutinei de tratare. Folosirea lui **signal()** nu este deci de dorit atunci cand se doreste scrierea de software care trebuie sa fie usor portabil pe alte variante de UNIX.

Urmatorul subparagraf va prezenta o metoda de captare a semnalelor care ofera mai multa flexibilitate decat cea descrisa anterior, prin folosirea apelului sistem **sigaction()** si a apelurilor asociate cu acesta.

2.2 Apelul sistem sigaction()

```
#include <signal.h>
```

```
int sigaction(int signal, const struct sigaction *act, struct sigaction *oldact);
```

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

```
int sigpending(sigset_t *set);
```

```
int sigsuspend(const sigset_t *mask);
```

Apelul sistem **sigaction()** este similar cu apelul sistem **signal()**. El are menirea de a defini comportarea procesului la primirea unui semnal. Acest apel primeste ca parametri numarul semnalului (signal), si doua structuri de tip *struct sigaction ** (act si oldact). Executia sa va avea ca efect instalarea noii actiuni pentru semnalul specificat din act (daca acest parametru este nenul), si salvarea actiunii curente in oldact (la fel, daca parametrul este nenul). Structura de tip *struct sigaction* este definita in felul urmator:

```
struct sigaction {  
    void (*sa_handler)(int);  
    void (*sa_sigaction)(int, siginfo_t *, void *);  
    sigset_t sa_mask;  
    int sa_flags;  
    void (*sa_restorer)(void);  
}
```

Parametrul sa_handler reprezinta noua rutina de tratare a semnalului specificat, similar cu parametrul handler prezentat la **signal()**. Alternativ, daca in sa_flags este setat indicatorul **SA_SIGINFO** (prezentat mai jos), se poate defini o rutina care primeste trei parametri in loc de unul singur, si in acest caz ea se specifica prin parametrul sa_sigaction. Pentru mai multe detalii legate de folosirea acestei din urma modalitati, consultati pagina de manual **sigaction(2)**. In unele cazuri, acesti doi parametri sunt prinsi intr-un **union**, si deci se recomanda sa se specifice doar unul din ei.

Parametrul sa_mask va specifica setul de semnale care vor fi blocate in timpul executiei rutinei de tratare a semnalului dat. Acest parametru este de tipul *sigset_t*, care este de fapt o masca de biti, cu cate un bit pentru fiecare semnal definit in sistem. Operatiile asupra acestui tip de masca se fac folosind functiile din familia **sigsetops(3)**. Sintaxa si functionarea acestor apeluri este foarte simpla, consultati pagina de manual pentru a vedea exact care sunt parametrii lor si modul de functionare.

Parametrul sa_flags va specifica un set de indicatori care afecteaza comportarea procesului de tratare a semnalelor. Acest parametru se formeaza prin efectuarea unei operatii de SAU logic folosind una sau mai multe din urmatoarele valori:

- **SA_NOCLDSTOP** - daca signal este **SIGCHLD**, procesul nu va primi un semnal **SIGCHLD** atunci cand procesul fiu este suspendat (de exemplu cu **SIGSTOP**), ci numai cand acesta isi termina executia;
- **SA_ONESHOT** sau **SA_RESETHAND** - va avea ca efect resetarea rutinei de tratare a semnalului la **SIG_DFL** dupa prima rulare a rutinei, asemanator cu comportamentul implementarii originale a apelului **signal()**;
- **SA_ONSTACK** - executia rutinei de tratare va avea loc folosind alta stiva;
- **SA_RESTART** - ofera compatibilitate cu comportamentul semnalelor in sistemele din familia 4.3BSD;
- **SA_NOMASK** sau **SA_NODEFER** - semnalul in discutie nu va fi inclus in mod automat in sa_mask (comportamentul implicit este acela de a impiedica aparitia unui semnal in timpul executiei rutinei de tratare a semnalului respectiv);
- **SA_SIGINFO** - se specifica atunci cand se doreste utilizarea lui sa_siginfo in loc de sa_handler. Pentru mai multe detalii, consultati pagina de manual **sigaction(2)**.

Apelul **sigprocmask()** este folosit pentru a modifica lista semnalelor care sunt blocate la un moment dat. Acest lucru se face in functie de valoarea parametrului how, in felul urmator:

- **SIG_BLOCK** - adauga la lista semnalelor blocate semnalele din lista set data ca parametru;
- **SIG_UNBLOCK** - sterge din lista semnalelor blocate semnalele aflate in lista set;
- **SIG_SETMASK** - face ca doar semnalele din lista set sa se regaseasca in lista semnalelor blocate.

Daca parametrul oldset este nenul, in el se va memora valoarea listei semnalelor blocate anterioara executiei lui **sigprocmask()**.

Apelul sistem **sigpending()** permite examinarea semnalelor care au aparut in timpul in care ele au fost blocate, prin returnarea acestor semnale in masca set data ca parametru.

Apelul **sigsuspend()** inlocuieste temporar masca semnalelor blocate a procesului cu cea data in parametrul mask si suspenda procesul pana la primirea unui semnal.

Apelurile **sigaction()**, **sigprocmask()** si **sigpending()** intorc valoarea 0 in caz de succes si -1 in caz de eroare. Apelul **sigsuspend()** intoarce intotdeauna -1, iar in mod normal variabila **errno** este setata la valoarea **EINTR**.

In urma executiei cu eroare a unuia din apelurile de mai sus, variabila **errno** poate sa ia una din urmatoarele valori:

- **EINVAL** - atunci cand a fost specificat un semnal invalid, adica nedefinit in implementarea curenta, sau unul din semnalele **SIGKILL** sau **SIGSTOP**;
- **EFAULT** - atunci cand una din variabilele date ca parametru indica spre o zona de memorie care nu face parte din spatiul de adrese al procesului;
- **EINTR** - atunci cand apelul a fost intrerupt.

3. Alte functii pentru lucrul cu semnale

3.1 Apelul sistem kill()

```
#include <sys/types.h>
```

```
#include <signal.h>
```

```
int kill(pid_t pid, int sig);
```

Acest apel sistem este folosit pentru a trimite un semnal unui anumit proces sau grup de procese.

In functie de valoarea parametrului pid, executia apelului va avea unul din urmatoarele efecte:

- Daca pid > 0, semnalul va fi trimis procesului care are PID-ul egal cu pid;

- Daca pid == 0, semnalul va fi trimis tuturor proceselor din acelasi grup de procese cu procesul curent;
- Daca pid == -1, semnalul va fi trimis tuturor proceselor care ruleaza in sistem (de notat faptul ca, in majoritatea implementarilor, nu se poate trimite in acest fel catre procesul **init** un semnal pentru care acesta nu are prevazuta o rutina de tratare, si de asemenea, faptul ca de obicei in acest fel procesului curent nu i se trimite semnalul respectiv);
- Daca pid < -1, se va trimite semnalul catre toate procesele din grupul de procese cu numarul -pid.

Daca valoarea lui sig este zero, nu se va trimite nici un semnal, dar apelul va executa verificarile de eroare. Acest lucru este util in cazul in care se doreste sa se stie, de exemplu, daca avem suficiente permisiuni pentru a trimite un semnal catre un proces dat.

Acest apel returneaza 0 in caz de succes si -1 in caz de eroare, setand variabila **errno** la una din urmatoarele valori in cazul executiei cu eroare:

- **EINVAL** - in cazul specificarii unui semnal invalid;
- **ESRCH** - in cazul in care procesul sau grupul de procese specificat nu exista;
- **EPERM** - in cazul in care nu se dispune de permisiuni suficiente pentru a trimite semnalul respectiv procesului specificat.

3.2 Functia raise()

#include <signal.h>

int raise(int sig);

Aceasta functie este folosita pentru a trimite un semnal catre procesul curent. Executia ei este similara cu executia urmatorului apel:

kill(getpid(), sig)

Functia returneaza 0 in caz de succes, si o valoare diferita de zero in caz de eroare.

3.3 Functia abort()

#include <stdlib.h>

void abort(void);

Aceasta functie are ca efect trimiterea catre procesul curent a unui semnal **SIGABRT**, care are ca efect terminarea anormala a procesului, mai putin daca semnalul este tratat de o rutina care nu se termina. Daca executia lui **abort()** are ca efect terminarea procesului, toate fisierele deschise in interiorul acestuia vor fi inchise.

Este important de notat ca daca semnalul **SIGABRT** este ignorat sau blocat, executia acestei functii nu va tine cont de acest lucru si procesul va fi terminat in mod anormal.

3.4 Functia alarm()

#include <unistd.h>

unsigned int alarm(int seconds);

Executia acestei functii are ca rezultat trimiterea, dupa scurgerea unui numar de secunde dat de seconds, a unui semnal de tipul **SIGALRM** catre procesul curent. Daca o alarma a fost deja programata, ea este anulata in momentul executiei ultimului apel, iar daca valoarea lui seconds este zero, nu va fi programata o alarma noua.

In urma executiei, se returneaza numarul de secunde ramase din alarma precedenta, sau 0 daca nu era programata nici o alarma.

4. Exemplu

Programul evidentiaza modul de utilizare a functiei **signal()** si comunicarea intre procese folosind semnale. Programul este format din doua procese, parinte si fiu. Parintele (**procesul a**)

numara continuu incepand de la zero, pana in momentul in care este intrerupt de catre utilizator. Intreruperea se face generand catre proces semnalul **SIGINT**, in mod explicit (folosind comanda **kill**) sau implicit (apasand Ctrl-C in consola in care programul ruleaza lansat in foreground). Pentru vizualizarea optima a rezultatelor, la fiecare pas procesul apeleaza functia **usleep()** generand astfel o intarziere de cate 1000 microsecunde.

De fiecare data cand procesul **a** ajunge cu numertoarea la un numar divizibil cu 10, el trimite un semnal **SIGUSR1** catre procesul **b**(fiul). Acesta trateaza semnalul **SIGUSR1** afisand pe ecran un mesaj. Daca parintele a fost intrerupt (de la tastatura, prin Ctrl-C, de exemplu), il informeaza pe fiu de aparitia acestui eveniment trimitandu-i semnalul **SIGUSR2**. Fiul primeste aceasta notificare si se termina, afisand un mesaj. Parintele ii preia in acest moment starea, dupa care se incheie si el.

Este necesar sa facem cateva observatii importante asupra implementarii de mai jos. In primul rand, programul defineste cate o functie de tratare pentru fiecare eveniment relevant:

- Procesul **a** asociaza semnalului **SIGINT** functia *process_a_ends()*. Conform codului acestei functii, in momentul primirii semnalului **SIGINT**, procesul va trimite un semnal **SIGUSR2** catre procesul **b**, dupa care apeleaza functia **wait()** , asteptand terminarea acestuia.
- Procesul **b** defineste doua functii de tratare: pentru **SIGUSR1** (functia *process_b_writes()*) si pentru **SIGUSR2** (functia *process_b_ends()*). Comportamentul acestor functii este banal, reducandu-se la afisarea unui mesaj si, in al doilea caz, la terminarea procesului prin **exit()**.

In al doilea rand, se observa ca procesul **b** ignora semnalul **SIGINT**. Motivul pentru care s-a decis acest lucru este pentru a-l face imun la actiunea implicita pe care acest semnal ar putea sa o aiba asupra procesului. Un efect nedorit ar fi cazul in care utilizatorul apasa Ctrl-C in consola din care a lansat programul, generand astfel **SIGINT**. Daca semnalul nu ar fi ignorat, procesul **b** s-ar termina inainte de a avea ocazia sa primeasca **SIGUSR2** de la parinte, deci terminarea lui nu ar respecta scenariul pe care si l-a propus sa-l implementeze aceasta aplicatie (efectul vizibil ar fi ca mesajul "Process b ends" nu ar mai aparea pe ecran).

In continuare, trebuie remarcat faptul ca, la inceputul programului principal, chiar inainte de crearea procesului fiu, sunt ignorate semnalele **SIGUSR1** si **SIGUSR2**. Aceasta secventa de cod poate parea putin importanta, dar necesitatea ei este justificata de efectele mai greu de prevazut pe care le are executia paralela a proceselor. Pentru a inelege explicatia, trebuie facuta mai intai precizarea ca, in Linux, dupa apelul functiei **fork()**, procesul fiu este inializat cu o copie a asocierilor semnal - functie de tratare ale praintelui. Daca semnalele **SIGUSR1** si **SIGUSR2** nu ar fi ignorate inca inainte de crearea procesului fiu, s-ar putea intampla urmatorul fenomen: procesele **a** si **b** sunt create intr-un timp foarte scurt, iar procesul **a** ajunge sa intre in executie inainte de **b**. El numara rapid si ajunge la un multiplu de 10 inainte ca procesul **b** sa isi armeze functiile de tratare pentru semnale. Procesul **a** trimite catre **b** semnalul **SIGUSR1**, dar cum acesta nu a declarat inca nici o functie pentru tratarea acestui semnal, sistemul de operare va proceda prin aplicarea comportamentului implicit al semnalului, adica terminarea procesului **b**. Se observa, deci, ca procesul **b** se incheie in mod eronat, inainte de a isi indeplini scopul pentru care a fost proiectat. Daca, insa, semnalul **SIGUSR1** este deja ignorat, in cel mai rau caz procesul **b** pierde cateva notificari de la **a**, lucru care, avand in vedere executia in paralel a proceselor, poate fi acceptat in scenariul nostru.

O ultima observatie este ca rezultatul functiei **fork()** este atribuit in program unei variabile globale *child_pid* pentru ca aceasta valoare sa poata fi accesibila ambelor functii de tratare a

semnalelor din procesul **a**. (Procesul **a** fiind parinte, valoarea acestei variabile va fi chiar identificatorul de proces al lui **b**).

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid = 0;
int n = 0;

void process_a_ends(int sig)
{
    int status;

    if (kill(child_pid, SIGUSR2) < 0)
    {
        printf("Error sending SIGUSR2 to child\n");
        exit(2);
    }

    /* waiting for the child to end */

    wait(&status);
    printf("Child ended with code %d\n", WEXITSTATUS(status));

    printf("Process a ends.\n");
    exit(0);
}

void process_a()
{
    int i;

    if (signal(SIGINT, process_a_ends) == SIG_ERR)
    {
        printf("Error setting handler for SIGTERM\n");
        exit(1);
    }
    for (i = 0;;i++)
    {
        usleep(1000);
        if (i%10 == 0)
            if (kill(child_pid, SIGUSR1) < 0)
            {
                printf("Error sending SIGUSR1 to child\n");
                exit(2);
            }
    }
}

void process_b_writes(int sig)
{
    printf("Process b received SIGUSR1: %d\n", ++n);
}
```

```

void process_b_ends(int sig)
{
    printf("Process b ends.\n");
    exit(0);
}

void process_b()
{
    /* Ignoring SIGINT. Process b will end only when receives SIGUSR2 */

    if (signal(SIGINT, SIG_IGN) == SIG_ERR)
    {
        printf("Error ignoring SIGINT in process b\n");
        exit(3);
    }

    /* Setting the signal handlers */

    if (signal(SIGUSR1, process_b_writes) == SIG_ERR)
    {
        printf("Error setting handler for SIGUSR1\n");
        exit(4);
    }
    if (signal(SIGUSR2, process_b_ends) == SIG_ERR)
    {
        printf("Error setting handler for SIGUSR2\n");
        exit(5);
    }

    /* Infinite loop;
       process b only responds to signals */

    while(1)
        ;
}

int main()
{
    int status;

    /* First, ignore the user signals,
       to prevent interrupting the child
       process before setting the
       appropriate handlers */

    signal(SIGUSR1, SIG_IGN);
    signal(SIGUSR2, SIG_IGN);

    /* Creating the child process.
       A global variable
       is used to store the child process ID
       in order to be able to use it from the
       signal handlers */

    if ((child_pid = fork()) < 0)
    {

```



```

        printf("Error creating child process\n");
        exit(1);
    }

    if (child_pid == 0) /* the child process */
    {
        process_b();
        exit(0);
    }
    else /* the parent process */
    {
        process_a();
    }

    /* this is still the parent code */

    return 0;
}

```



Scrieti un program UNIX in C care realizeaza urmatoarele:

- Sistemul este format din doua procese: parinte si fiu
- Fiul ignora semnalul SIG_TERM
- Parintele instaleaza o alarma care il va intrerupe dupa 7 secunde; intre timp, va afisa pe ecran, in mod continuu, caracterul 'A'. La primirea semnalului de alarma va anunta fiul prin semnalul SIGUSR1, dar va continua sa afiseze (la infinit) caracterul 'A'
- Fiul afiseaza continuu caracterul 'B', pana in momentul in care primeste semnalul SIGUSR1, moment in care afiseaza un mesaj si se termina
- Cand fiul se termina, parintele preia starea lui si se termina si el

Vor fi folosite functii de tratare a semnalelor (una pentru generarea SIGUSR1 in parinte si doua pentru terminarea proceselor parinte si fiu) si functiile alarm(), kill(). Indicatie: in UNIX, cand un proces fiu se termina, parintele primeste automat semnalul SIGCHLD.

Nota: Pentru captarea semnalelor se va folosi sigaction(). Daca se doreste, se poate realiza in plus si o varianta csignal().

LAB 4

Comunicarea între procese: Pipe

1. Scopul lucrării

O metoda foarte des utilizata in UNIX pentru comunicarea între procese este folosirea primitivei numita *pipe* (conducta). "Conducta" este o cale de legatura care poate fi stabilita între doua procese înrudite (au un stramos comun sau sunt în relatia stramos-urmas). Ea are doua capete, unul prin care se pot scrie date si altul prin care datele pot fi citite, permitand o comunicare într-o singura directie. În general, sistemul de operare permite conectarea a unuia sau mai multor procese la fiecare din capetele unui *pipe*, astfel încat, la un moment dat este posibil sa existe mai multe procese care scriu, respectiv mai multe procese care citesc din *pipe*. Se realizeaza, astfel, comunicarea unidirectionala între procesele care scriu si procesele care citesc.

2. Considerații teoretice

2.1. Apelul sistem *pipe()*

Crearea conductelor de date de face în UNIX folosind apelul sistem *pipe()*:

```
int pipe(int fildes[2]);
```

Functia creeaza un *pipe*, precum si o pereche de descriptori de fisier care refera cele doua capete ale acestuia. Descriptorii sunt returnati catre programul apelant completandu-se cele doua pozitii ale tabloului *fildes* trimis ca parametru apelului sistem. Pe prima pozitie va fi memorat descriptorul care indica extremitatea prin care se pot citi date (capatul de citire), iar pe a doua pozitie va fi memorat descriptorul capatului de scriere în *pipe*.

Cei doi descriptori sunt descriptori de fisier obisnuiti, asemanatori celor returnati de apelul sistem *open()*. Mai mult, *pipe*-ul poate fi folosit în mod similar folosirii fisierelor, adica în el pot fi scrise date folosind functia *write()* (aplicata capatului de scriere) si pot fi citite date prin functia *read()* (aplicata capatului de citire).

Fiind implicati descriptori de fisier obisnuiti, daca un *pipe* este creat într-un proces parinte, fiii acestuia vor mosteni cei doi descriptori (asa cum, în general, ei mostenesc orice descriptor de fisier deschis de parinte). Prin urmare, atat parintele cat si fiii vor putea scrie sau citi din *pipe*. În acest mod se justifica afirmatia facuta la inceputul acestui document prin care se spunea ca *pipe*-urile sunt folosite la comunicarea între procese *inrudite*. Pentru ca legatura dintre procese sa se faca corect, fiecare proces trebuie sa declare daca va folosi *pipe*-ul pentru a scrie în el (transmitand informatii altor procese) sau îl va folosi doar pentru citire. În acest scop, fiecare proces trebuie sa închida capatul *pipe*-ului pe care nu îl foloseste: procesele care scriu în *pipe* vor închide capatul de citire, iar procesele care citesc vor închide capatul de scriere, folosind functia *close()*.

Functia returneaza 0 daca operatia de creare s-a efectuat cu succes si -1 în caz de eroare.

Un posibil scenariu pentru crearea unui sistem format din doua procese care comunica prin pipe este urmatorul:

- procesul parinte creeaza un *pipe*
- parintele apeleaza *fork()* pentru a crea fiul
- fiul inchide unul din capete (ex: capatul de citire)
- parintele inchide celalalt capat al *pipe*-ului (cel de scriere)
- fiul scrie date in *pipe* folosind descriptorul ramas deschis (capatul de scriere)
- parintele citeste date din *pipe* prin capatul de citire.

Primitiva *pipe* se comporta in mod asemanator cu o structura de date coada: scrierea introduce elemente in coada, iar citirea le extrage pe la capatul opus.

Iata in continuare o portiune de program scris conform scenariului de mai sus:

```
void main()
{
    int pfd[2];
    int pid;
    ...
    if(pipe(pfd)<0)
    {
        printf("Eroare la crearea pipe-ului\n");
        exit(1);
    }
    ...
    if((pid=fork())<0)
    {
        printf("Eroare la fork\n");
        exit(1);
    }
    if(pid==0) /* procesul fiu */
    {
        close(pfd[0]); /* inchide capatul de citire; */
                        /* procesul va scrie in pipe */

        ...
        write(pfd[1],buff,len); /* operatie de scriere in pipe */
        ...
        close(pfd[1]); /* la sfarsit inchide si capatul utilizat */
        exit(0);
    }
    else /* procesul parinte */
    {
        close(pfd[1]); /* inchide capatul de scriere; */
                        /* procesul va citi din pipe */

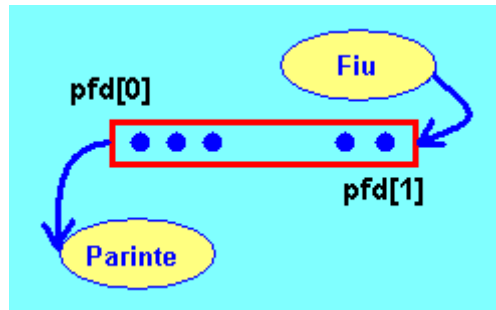
        ...
        read(pfd[0],buff,len); /* operatie de citire din pipe */
        ...
        close(pfd[0]); /* la sfarsit inchide si capatul utilizat */
        exit(0);
    }
}
```

```

    }
}

```

Conexiunea între cele două procese din care este format programul de mai sus este reflectată în figura următoare:



Observatii:

1. Cantitatea de date care poate fi scrisă la un moment dat într-un *pipe* este limitată. Numărul de octeți pe care un *pipe* îi poate păstra fără ca ei să fie extrasi prin citire de către un proces este dependentă de sistem (de implementare). Standardul POSIX specifică limita minimă a capacității unui *pipe*: 512 octeți. Atunci când un *pipe* este "plin", operația *write()* se va bloca până când un alt proces nu citește suficienți octeți din *pipe*.
2. Un proces care citește din *pipe* va primi valoarea **0** ca valoare returnată de *read()* în momentul în care toate procesele care scriau în *pipe* au închis capatul de scriere și nu mai există date în *pipe*.
3. Dacă pentru un *pipe* sunt conectate procese doar la capatul de scriere (cele de la capatul opus au închis toate conexiunile) operațiile *write* efectuate de procesele rămase vor returna eroare. Intern, în această situație va fi generat semnalul SIG_PIPE care va întrerupe apelul sistem *write* respectiv. Codul de eroare (setat în variabila globală *errno*) rezultat este cel corespunzător mesajului de eroare "Broken pipe".
4. Operația de scriere în *pipe* este atomică doar în cazul în care numărul de octeți scrși este mai mic decât constanta PIPE_BUF. Altfel, în șirul de octeți scrși pot fi intercalate datele scrșe de un alt proces în *pipe*. Pentru detalii, consultați manualul *pipe(7)*.

Exemplu:

```

#include <unistd.h>
int main()
{
    int pfd[2];
    int pid;
    char buff[30]="hello";
    if(pipe(pfd)<0)
    {
        printf("Eroare la crearea pipe-ului\n");
        exit(1);
    }

    if((pid=fork())<0)
    {
        printf("Eroare la fork\n");
        exit(1);
    }
    if(pid==0) /* procesul fiu */
    {

```

```

        close(pfd[0]); /* inchide capatul de citire; */
        /* procesul va scrie in pipe */
        write(pfd[1],buff,5); /* operatie de scriere in pipe */
        close(pfd[1]); /* la sfarsit inchide si capatul utilizat */
        exit(0);
    }
    else /* procesul parinte */
    {
        close(pfd[1]); /* inchide capatul de scriere; */
        /* procesul va citi din pipe */
        read(pfd[0],buff,5); /* operatie de citire din pipe */
        printf ("am primit: %s", buff);
        close(pfd[0]); /* la sfarsit inchide si capatul utilizat */
        exit(0);
    }
}

```



1. Sa se scrie un program care creeaza trei procese, astfel:
 - primul proces (parinte) citeste dintr-un fisier cu numele *date.txt* un sir de caractere, pana la sfarsitul fisierului si le trimite printr-un *pipe* primului fiu
 - primul fiu primește caracterele de la parinte si selecteaza din ele literele mici, pe care le trimite printr-un *pipe* catre cel de-al doilea fiu.
 - al doilea fiu creeaza un fisier numit *statistica.txt*, in care va memora, pe cate o linie, fiecare litera distincta intalnita si numarul de aparitii a acesteia in fluxul de date primit. In final, va trimite catre parinte, printr-un *pipe* suplimentar, numarul de litere distincte intalnite.
 - parintele afiseaza pe ecran rezultatul primit de la al doilea fiu.
2. Sa se realizeze un program de tip *producatori-consumatori* astfel:
 - programul primește ca parametri in linia de comanda doua numere: numarul de procese producator si numarul de procese consumator;
 - resursa comuna tuturor proceselor este un *pipe* creat de procesul parinte;
 - parintele este la randul lui producator;
 - in general, producatorii scriu in *pipe* un numar oarecare de caractere, iar consumatorii citesc din *pipe*, caracter cu caracter, cat timp acest lucru este posibil.

Producatorii vor avea urmatoarea forma:

- a) Parintele produce un numar de caractere '*';
- b) Ceilalti producatori sunt procese independente (existente pe disc ca programe de sine statatoare) care genereaza la iesirea standard un numar oarecare de caractere '#'. Pentru realizarea scopului propus, iesirea standard a acestor procese va fi conectata la capatul de scriere al *pipe*-ului.

- c) Cel puțin unul dintre producatori este comanda 'ls'.

Consumatorii citesc caracterele din *pipe* si isi afiseaza identificatorul de proces urmat de caracterul citit (la un moment dat).

In sistem mai exista un *pipe* prin care consumatorii vor raporta in final rezultatele catre parintele tuturor, scriind in el, cu ajutorul functiei *fprintf()*, linii de forma:

numar_proces : numar_octeti

unde *numar_octeti* este numarul total de octeti cititi de procesul respectiv din *pipe*. Parintele va prelua aceste informatii si le va afisa pe ecran.